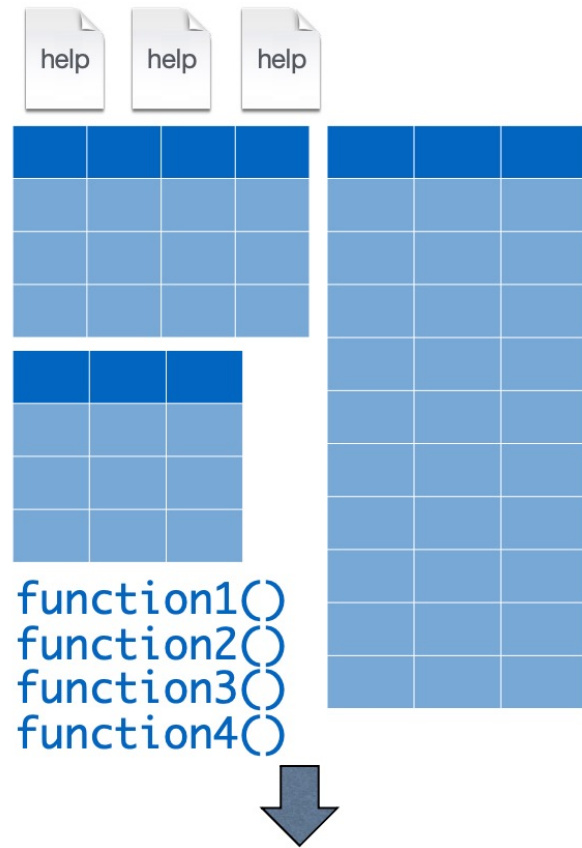


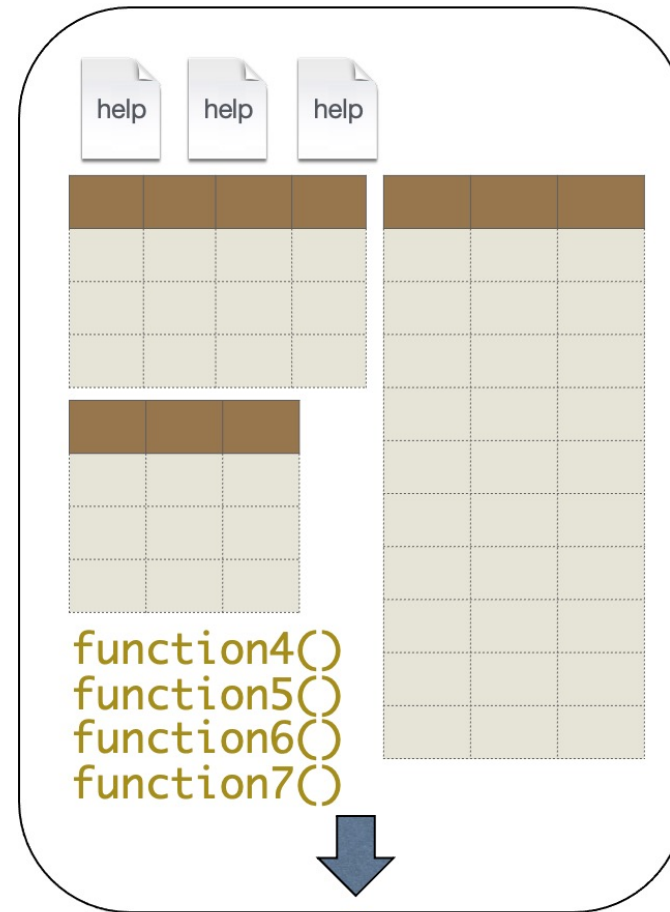
Tidyverse in R

Aakrosh Ratan

R packages

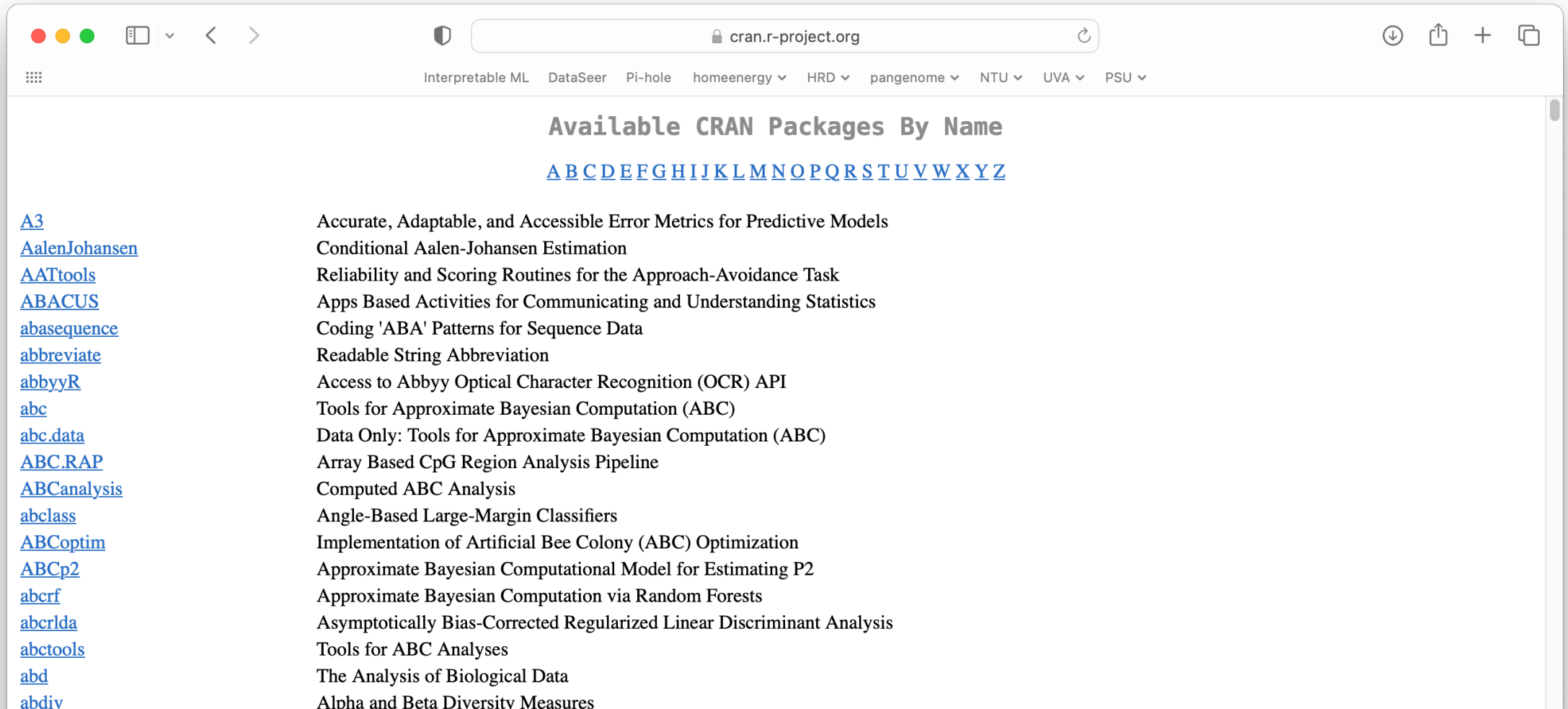


“base R”



R “Package” 

Search and you shall find (in most cases)

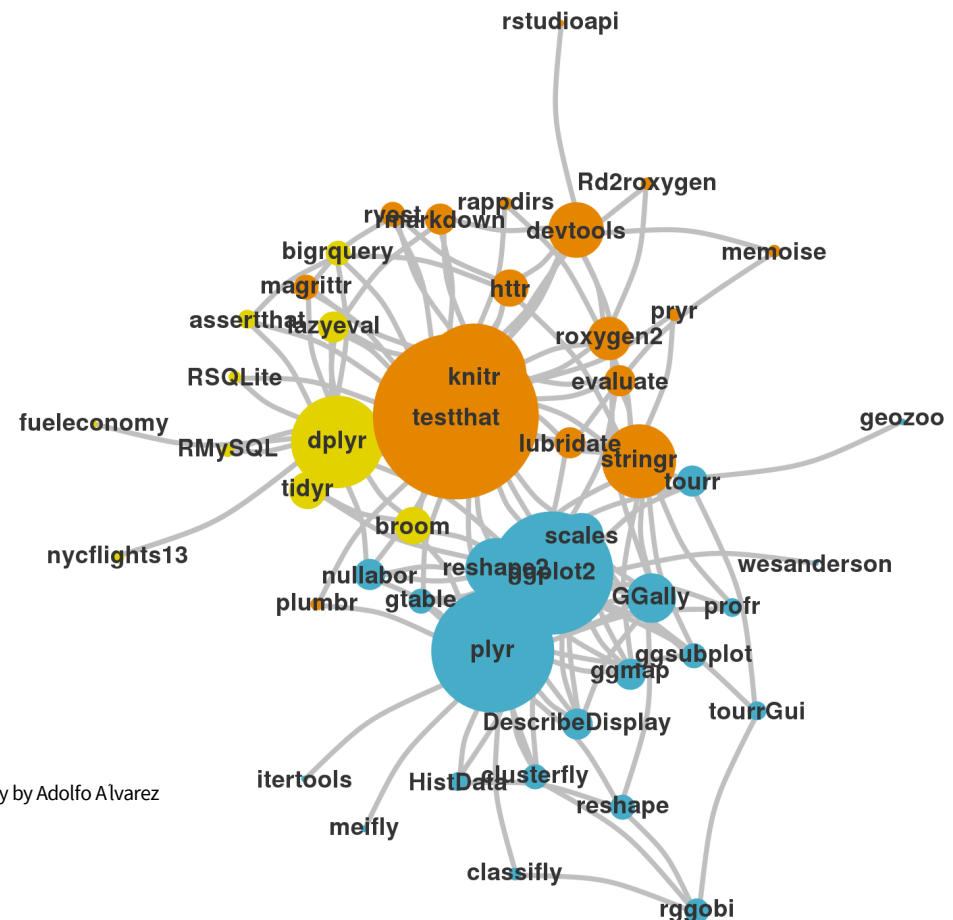


The screenshot shows a web browser window with the URL cran.r-project.org. The page title is "Available CRAN Packages By Name". Below the title is a navigation bar with links for each letter of the alphabet: [A](#) [B](#) [C](#) [D](#) [E](#) [F](#) [G](#) [H](#) [I](#) [J](#) [K](#) [L](#) [M](#) [N](#) [O](#) [P](#) [Q](#) [R](#) [S](#) [T](#) [U](#) [V](#) [W](#) [X](#) [Y](#) [Z](#). The main content area lists packages starting with 'A'. On the left side of the page, there is a vertical list of package names, each preceded by a blue underlined link to its page.

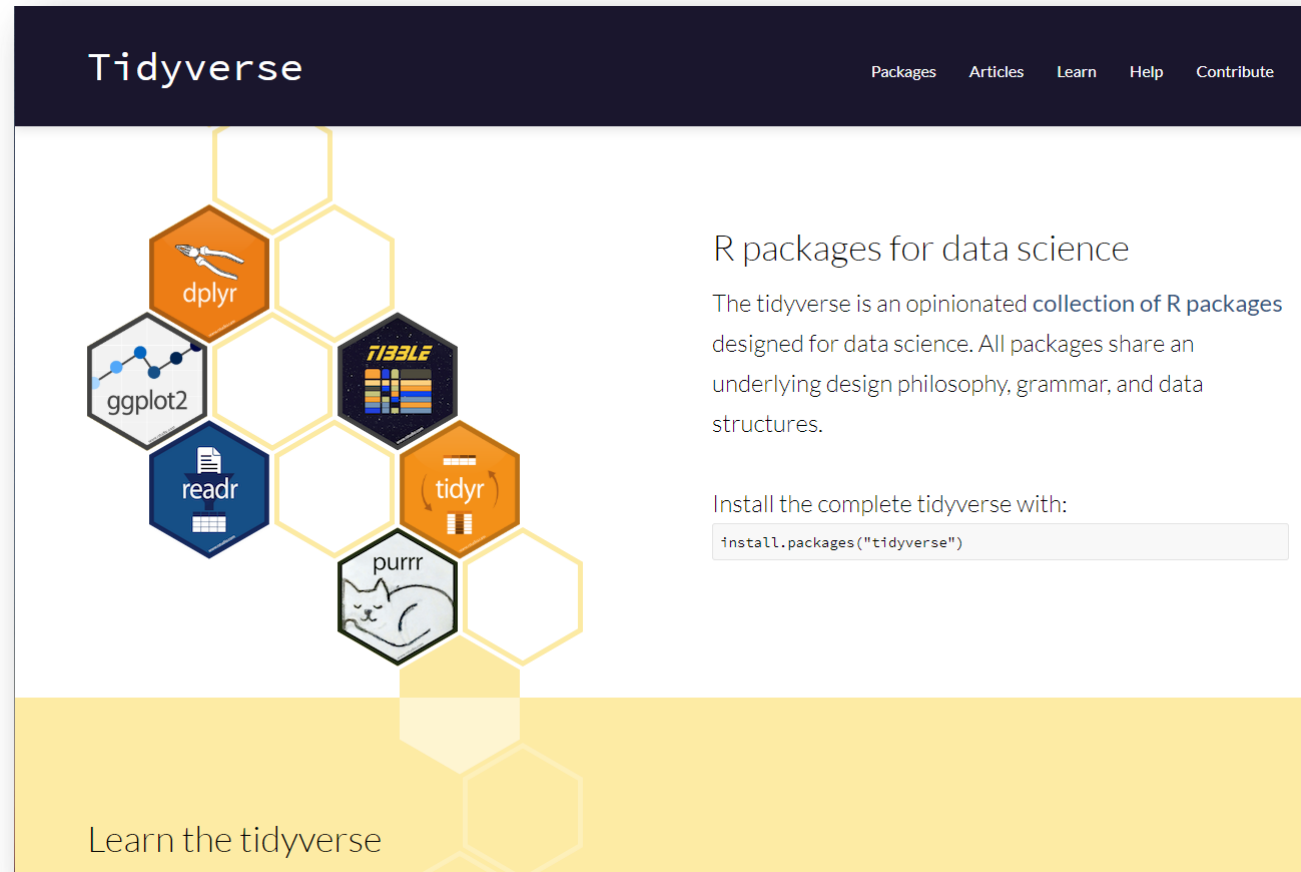
Package Name	Description
A3	Accurate, Adaptable, and Accessible Error Metrics for Predictive Models
AalenJohansen	Conditional Aalen-Johansen Estimation
AATtools	Reliability and Scoring Routines for the Approach-Avoidance Task
ABACUS	Apps Based Activities for Communicating and Understanding Statistics
abasequence	Coding 'ABA' Patterns for Sequence Data
abbreviate	Readable String Abbreviation
abbyyR	Access to Abbyy Optical Character Recognition (OCR) API
abc	Tools for Approximate Bayesian Computation (ABC)
abc.data	Data Only: Tools for Approximate Bayesian Computation (ABC)
ABC.RAP	Array Based CpG Region Analysis Pipeline
ABCanalysis	Computed ABC Analysis
abclass	Angle-Based Large-Margin Classifiers
ABCOptim	Implementation of Artificial Bee Colony (ABC) Optimization
ABCp2	Approximate Bayesian Computational Model for Estimating P2
abcrf	Approximate Bayesian Computation via Random Forests
abcrlda	Asymptotically Bias-Corrected Regularized Linear Discriminant Analysis
abctools	Tools for ABC Analyses
abd	The Analysis of Biological Data
abdiv	Alpha and Beta Diversity Measures

The Tidyverse

A collection of modern R packages that share common philosophies, embed best practices, and are designed to work together.



tidyverse.org



The screenshot shows the tidyverse.org website. At the top is a dark blue header with the 'Tidyverse' logo on the left and navigation links 'Packages', 'Articles', 'Learn', 'Help', and 'Contribute' on the right. The main content area features a large graphic on the left consisting of a cluster of hexagons, some of which contain icons for R packages: 'dplyr' (orange with a scuba diver), 'ggplot2' (grey with a line graph), 'readr' (blue with a document icon), 'tidyr' (orange with a document icon), 'purrr' (grey with a cat icon), and 'TIBBLE' (dark blue with a grid icon). To the right of this graphic, the text reads 'R packages for data science' followed by a paragraph: 'The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures.' Below this is the instruction 'Install the complete tidyverse with:' followed by a code block containing the command `install.packages("tidyverse")`. At the bottom of the page, a yellow banner contains the text 'Learn the tidyverse'.

Tidyverse

Packages Articles Learn Help Contribute

R packages for data science

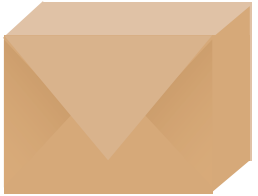
The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

Install the complete tidyverse with:

```
install.packages("tidyverse")
```

Learn the tidyverse

tidyverse



An R package that serves as a short cut for installing and loading the components of the tidyverse.

```
library("tidyverse")
```

R Data Structures

Vector

- 1D Data Structure of fixed type

	scores
1	0.8
2	1.2
3	3.3
4	1.8
5	2.7

`mean(scores)`
`sd(scores)`

List

- Collection of vectors

```
results$counts  
mean(results$counts)
```

results	
"ratios"	"counts"
1	2
1 0.8	1 100
2 1.2	2 300
3 3.3	3 200
4 1.8	
5 2.7	

Data Frame

- Collection of vectors with same lengths
- Gain the concept of 'rows'

```
all.results$mon  
mean(all.results$mon)
```

all.results				
	"mon"	"tue"	"wed"	"pass"
	1	2	3	4
1	0.8	0.9	0.8	T
2	0.6	0.7	0.5	F
3	0.2	0.3	0.3	F
4	0.8	0.8	0.9	T
5	0.6	1.0	0.9	T

Tibble

- Collection of vectors with same lengths
- Gain the concept of 'rows'

```
all.results$mon  
mean(all.results$mon)
```

all.results				
	"mon"	"tue"	"wed"	"pass"
	1	2	3	4
1	0.8	0.9	0.8	T
2	0.6	0.7	0.5	F
3	0.2	0.3	0.3	F
4	0.8	0.8	0.9	T
5	0.6	1.0	0.9	T

Tibbles are nicer dataframes

```
> head(as.data.frame(data))
```

	Probe	Chromosome	Start	End	Probe	Strand	Feature
1	AL645608.2	1	911435	914948		+	AL645608.2
2	LINC02593	1	916865	921016		-	LINC02593
3	SAMD11	1	923928	944581		+	SAMD11
4	TMEM51-AS1	1	15111815	15153618		-	TMEM51-AS1
5	TMEM51	1	15152532	15220478		+	TMEM51
6	FHAD1	1	15247272	15400283		+	FHAD1

	Description
1	novel transcript
2	long intergenic non-protein coding RNA 2593 [Source:HGNC Symbol;Acc:HGNC:53933]
3	sterile alpha motif domain containing 11 [Source:HGNC Symbol;Acc:HGNC:28706]
4	TMEM51 antisense RNA 1 [Source:HGNC Symbol;Acc:HGNC:26301]
5	transmembrane protein 51 [Source:HGNC Symbol;Acc:HGNC:25488]
6	forkhead associated phosphopeptide binding domain 1 [Source:HGNC Symbol;Acc:HGNC:25489]

Tibbles are nicer dataframes

```
> head(as_tibble(data))
```

```
# A tibble: 6 x 12
```

	Probe	Chromosome	Start	End	`Probe Strand`	Feature	ID	Description
	<chr>	<dbl>	<dbl>	<dbl>	<chr>	<chr>	<chr>	<chr>
1	AL64~	1	9.11e5	9.15e5	+	AL6456~	ENSG~	novel tran~
2	LINC~	1	9.17e5	9.21e5	-	LINC02~	ENSG~	long inter~
3	SAMD~	1	9.24e5	9.45e5	+	SAMD11	ENSG~	sterile al~
4	TMEM~	1	1.51e7	1.52e7	-	TMEM51~	ENSG~	TMEM51 ant~
5	TMEM~	1	1.52e7	1.52e7	+	TMEM51	ENSG~	transmembr~
6	FHAD1	1	1.52e7	1.54e7	+	FHAD1	ENSG~	forkhead a~

```
# ... with 4 more variables: `Feature Strand` <chr>, Type <chr>, `Feature
```

```
# Orientation` <chr>, Distance <dbl>
```

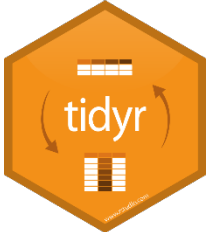
Tidyverse Packages



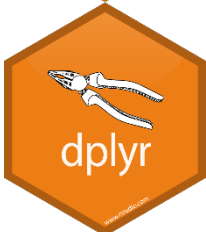
- Tibble - data storage



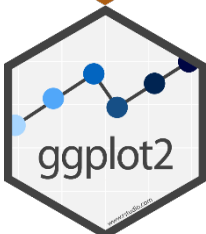
- ReadR - reading data from files



- Tidyr - Model data correctly



- Dplyr - Manipulate and filter data



- Ggplot2 - Draw figures and graphs

Installation and calling

- Once per machine (don't include in script)
 - `install.packages("tidyverse")`
- Once per R session (DO include in script)
 - `library(tidyverse)`

```
-- Attaching packages ----- tidyverse 1.3.1 --  
v ggplot2 3.3.3      v purrr  0.3.4  
v tibble  3.1.2      v dplyr  1.0.6  
v tidyr   1.1.3      v stringr 1.4.0  
v readr   2.0.0      v forcats 0.5.1
```

```
-- Conflicts ----- tidyverse_conflicts()  
x dplyr::filter() masks stats::filter()  
x dplyr::lag() masks stats::lag()
```



Reading and Writing Files with readr

- Provides functions to read from text files into tibbles or write from tibbles to text files
 - `data <- read_csv("file.csv")`
 - `data <- read_tsv("file.tsv")`
 - `data <- read_delim("file.tsv", delim="\t")`
 - `write_csv(data, "file.csv")`
 - `write_tsv(data, "file.csv")`



Reading and Writing Excel Files

- Provides functions to read from excel files into tibbles
 - `library(readxl)`
 - `data <- read_excel("file.xlsx")`



```
[> library(tidyverse)]
— Attaching packages — tidyverse 1.3.2 —
✓ ggplot2 3.3.6      ✓ purrr 0.3.4
✓ tibble 3.1.8       ✓ dplyr 1.0.10
✓ tidyr 1.2.1        ✓ stringr 1.4.1
✓ readr 2.1.2        ✓ forcats 0.5.2
— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag() masks stats::lag()
[> data <- read_tsv("PCa_expression_data.tsv")]
```

Rows: 327 Columns: 63

— Column specification —

Delimiter: "\t"

chr (2): Race, CAPRASgroup

dbl (61): ID, AN07, ASF1B, ASPM, AZGP1, BGN, BIRC5, BUB1B, C6orf10, CAMK2N1,...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message

.

```
[> data
```

A tibble: 327 × 63

	ID	Race	CAPRA... ¹	AN07	ASF1B	ASPM	AZGP1	BGN	BIRC5	BUB1B	C6orf10	CAMK2N1
	<dbl>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	AAM	INTERM...	10.9	4.11	3.78	13.3	10.6	5.95	4.59	3.78	10.7
2	2	AAM	LOW	10.5	6	5.64	13.8	11.5	6.64	5.64	3.41	9.7
3	3	AAM	LOW	11.2	4.44	4.63	12.5	10.2	6.63	5.44	3.21	9
4	4	AAM	LOW	11.4	4.88	4.19	14.3	10.2	5.54	4.4	3.66	9.65
5	5	AAM	LOW	10.5	3.56	4.37	13.0	10.3	4.84	4.56	2.98	9.71
6	6	AAM	LOW	10.9	5.98	5.2	14.8	10.9	7.17	5.85	5.06	8.97
7	7	AAM	INTERM...	7.54	5.36	4.18	12.4	11.6	5.7	5.03	3.44	10.7
8	8	AAM	LOW	11.4	3.68	3.82	14.5	10.3	5.16	5	3.68	10.2
9	9	AAM	LOW	9.48	6.15	5.96	15.2	12.1	7.52	5.88	3.92	9.75
10	10	AAM	INTERM...	10.3	4.75	4.7	14.3	11.1	6.13	4.33	3.11	8.68

... with 317 more rows, 51 more variables: CDC20 <dbl>, CDCA3 <dbl>,

CDK1 <dbl>, CDKN3 <dbl>, CENPF <dbl>, CENPM <dbl>, CEP55 <dbl>,

COL1A1 <dbl>, DLGAP5 <dbl>, DTL <dbl>, EPPK1 <dbl>, FAM13C <dbl>,

FLNC <dbl>, FOXM1 <dbl>, GSN <dbl>, GSTM2 <dbl>, IQGAP3 <dbl>,

KIAA0101 <dbl>, KIF11 <dbl>, KIF20A <dbl>, KLK2 <dbl>, LASP1 <dbl>,

MCM10 <dbl>, MYBPC1 <dbl>, NFIB <dbl>, NUSAP1 <dbl>, ORC6 <dbl>, PBK <dbl>,

PBX1 <dbl>, `PCAT-32` <dbl>, `PCAT-80` <dbl>, PCDH7 <dbl>, PLK1 <dbl>, ...

i Use `print(n = ...)` to see more rows, and `colnames()` to see all variable names

'Tidy' Data Format

- Tibbles give you a 2D data structure where each column must be of a fixed data type
- Often data can be put into this sort of structure in more than one way
- Is there a right / wrong way to structure your data?
- Tidyverse has an opinion!



Journal of Statistical Software

August 2014, Volume 59, Issue 10.

<http://www.jstatsoft.org/>

Tidy Data

Hadley Wickham
RStudio

Long vs Wide Data Modelling

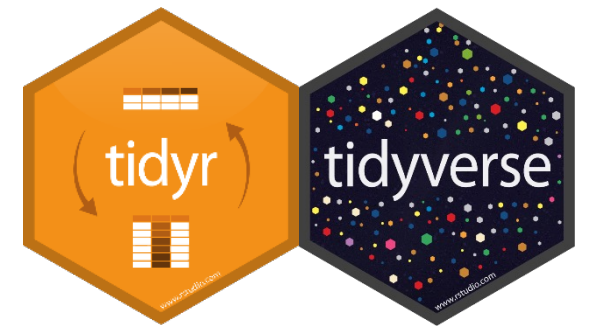
- Consider a simple experiment:
- Two genes tested (ABC1 and DEF1)
- Two conditions (WT and KO)
- Three replicates for each condition

Wide Format

Gene	WT_1	WT_2	WT_3	KO_1	KO_2	KO_3
ABC1	8.86	4.18	8.90	4.00	14.52	13.39
DEF1	29.60	41.22	36.15	11.18	16.68	1.64

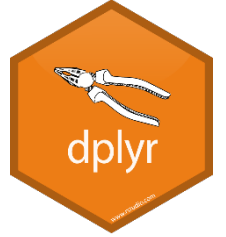
- Compact
- Easy to read
- Shows linkage for genes
- No explicit genotype or replicate
- Values spread out over multiple rows and columns
- Not extensible to more metadata

Long Format



Gene	Genotype	Replicate	Value
ABC1	WT	1	8.86
ABC1	WT	2	4.18
ABC1	WT	3	8.90
ABC1	KO	1	4.00
ABC1	KO	2	14.52
ABC1	KO	3	13.39
DEF1	WT	1	29.60
DEF1	WT	2	41.22
DEF1	WT	3	36.15
DEF1	KO	1	11.18
DEF1	KO	2	16.68
DEF1	KO	3	1.64

- More verbose (repeated values)
- Explicit genotype and replicate
- All values in a single column
- Extensible to more metadata



Filtering and subsetting

- Tidyverse (specifically dplyr) comes with functions to manipulate your data.
- All functions take a tibble as their first argument
- All functions return a modified tibble
 - Selecting columns
 - Logical subsetting

The data we're starting with

```
[> data
# A tibble: 327 x 63
  ID Race  CAPRA...1 AN07 ASF1B  ASPM AZGP1  BGN BIRC5 BUB1B C6orf10 CAMK2N1
  <dbl> <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1     1 AAM  INTERM... 10.9  4.11  3.78 13.3 10.6  5.95  4.59  3.78 10.7
2     2 AAM  LOW      10.5   6    5.64 13.8 11.5  6.64  5.64  3.41  9.7
3     3 AAM  LOW      11.2  4.44  4.63 12.5 10.2  6.63  5.44  3.21   9
4     4 AAM  LOW      11.4  4.88  4.19 14.3 10.2  5.54  4.4   3.66  9.65
5     5 AAM  LOW      10.5  3.56  4.37 13.0 10.3  4.84  4.56  2.98  9.71
6     6 AAM  LOW      10.9  5.98  5.2   14.8 10.9  7.17  5.85  5.06  8.97
7     7 AAM  INTERM... 7.54  5.36  4.18 12.4 11.6  5.7   5.03  3.44 10.7
8     8 AAM  LOW      11.4  3.68  3.82 14.5 10.3  5.16  5     3.68 10.2
9     9 AAM  LOW      9.48  6.15  5.96 15.2 12.1  7.52  5.88  3.92  9.75
10    10 AAM  INTERM... 10.3  4.75  4.7   14.3 11.1  6.13  4.33  3.11  8.68
# ... with 317 more rows, 51 more variables: CDC20 <dbl>, CDCA3 <dbl>,
# CDK1 <dbl>, CDKN3 <dbl>, CENPF <dbl>, CENPM <dbl>, CEP55 <dbl>,
# COL1A1 <dbl>, DLGAP5 <dbl>, DTL <dbl>, EPPK1 <dbl>, FAM13C <dbl>,
# FLNC <dbl>, FOXM1 <dbl>, GSN <dbl>, GSTM2 <dbl>, IQGAP3 <dbl>,
# KIAA0101 <dbl>, KIF11 <dbl>, KIF20A <dbl>, KLK2 <dbl>, LASP1 <dbl>,
# MCM10 <dbl>, MYBPC1 <dbl>, NFIB <dbl>, NUSAP1 <dbl>, ORC6 <dbl>, PBK <dbl>,
# PBX1 <dbl>, `PCAT-32` <dbl>, `PCAT-80` <dbl>, PCDH7 <dbl>, PLK1 <dbl>, ...
# i Use `print(n = ...)` to see more rows, and `colnames()` to see all variable
names
```

Using select to pick columns

```
> select(data, ID, Race, BIRC5)
# A tibble: 327 × 3
      ID Race  BIRC5
  <dbl> <chr> <dbl>
1     1 AAM   5.95
2     2 AAM   6.64
3     3 AAM   6.63
4     4 AAM   5.54
5     5 AAM   4.84
6     6 AAM   7.17
7     7 AAM   5.7
8     8 AAM   5.16
9     9 AAM   7.52
10    10 AAM   6.13
# ... with 317 more rows
# i Use `print(n = ...)` to see more rows
```

You can use positions instead of names

```
> select(data, 1, 2, 9)
# A tibble: 327 × 3
   ID Race  BIRC5
  <dbl> <chr> <dbl>
1     1 AAM    5.95
2     2 AAM    6.64
3     3 AAM    6.63
4     4 AAM    5.54
5     5 AAM    4.84
6     6 AAM    7.17
7     7 AAM    5.7
8     8 AAM    5.16
9     9 AAM    7.52
10    10 AAM    6.13
# ... with 317 more rows
# i Use `print(n = ...)` to see more rows
```

You can use negative selections

```
> select(data, -1, -3)
```

```
# A tibble: 327 × 61
```

	Race	AN07	ASF1B	ASPM	AZGP1	BGN	BIRC5	BUB1B	C6orf10	CAMK2N1	CDC20	CDCA3
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	AAM	10.9	4.11	3.78	13.3	10.6	5.95	4.59	3.78	10.7	5.11	3.78
2	AAM	10.5	6	5.64	13.8	11.5	6.64	5.64	3.41	9.7	5.64	5.15
3	AAM	11.2	4.44	4.63	12.5	10.2	6.63	5.44	3.21	9	5.99	1.63
4	AAM	11.4	4.88	4.19	14.3	10.2	5.54	4.4	3.66	9.65	5.3	3.66
5	AAM	10.5	3.56	4.37	13.0	10.3	4.84	4.56	2.98	9.71	5.73	3.79
6	AAM	10.9	5.98	5.2	14.8	10.9	7.17	5.85	5.06	8.97	6.39	4.51
7	AAM	7.54	5.36	4.18	12.4	11.6	5.7	5.03	3.44	10.7	6.2	4.73
8	AAM	11.4	3.68	3.82	14.5	10.3	5.16	5	3.68	10.2	5.22	3.53
9	AAM	9.48	6.15	5.96	15.2	12.1	7.52	5.88	3.92	9.75	6.82	4.21
10	AAM	10.3	4.75	4.7	14.3	11.1	6.13	4.33	3.11	8.68	5.87	3.64

```
# ... with 317 more rows, and 49 more variables: CDK1 <dbl>, CDKN3 <dbl>,
```

```
# CENPF <dbl>, CENPM <dbl>, CEP55 <dbl>, COL1A1 <dbl>, DLGAP5 <dbl>,
```

```
# DTL <dbl>, EPPK1 <dbl>, FAM13C <dbl>, FLNC <dbl>, FOXM1 <dbl>, GSN <dbl>,
```

```
# GSTM2 <dbl>, IQGAP3 <dbl>, KIAA0101 <dbl>, KIF11 <dbl>, KIF20A <dbl>,
```

```
# KLK2 <dbl>, LASP1 <dbl>, MCM10 <dbl>, MYBPC1 <dbl>, NFIB <dbl>,
```

```
# NUSAP1 <dbl>, ORC6 <dbl>, PBK <dbl>, PBX1 <dbl>, `PCAT-32` <dbl>,
```

```
# `PCAT-80` <dbl>, PCDH7 <dbl>, PLK1 <dbl>, PRC1 <dbl>, PTTG1 <dbl>, ...
```

```
# i Use `print(n = ...)` to see more rows, and `colnames()` to see all variable names
```


Functional selections using filter

```
> filter(data, Race == "AAM")
# A tibble: 95 × 63
   ID Race CAPRA...1 AN07 ASF1B ASPM AZGP1 BGN BIRC5 BUB1B C6orf10 CAMK2N1
  <dbl> <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1     1 AAM INTERM... 10.9  4.11  3.78 13.3 10.6  5.95  4.59  3.78 10.7
2     2 AAM LOW      10.5  6     5.64 13.8 11.5  6.64  5.64  3.41  9.7
3     3 AAM LOW      11.2  4.44  4.63 12.5 10.2  6.63  5.44  3.21  9
4     4 AAM LOW      11.4  4.88  4.19 14.3 10.2  5.54  4.4   3.66  9.65
5     5 AAM LOW      10.5  3.56  4.37 13.0 10.3  4.84  4.56  2.98  9.71
6     6 AAM LOW      10.9  5.98  5.2   14.8 10.9  7.17  5.85  5.06  8.97
7     7 AAM INTERM... 7.54  5.36  4.18 12.4 11.6  5.7   5.03  3.44 10.7
8     8 AAM LOW      11.4  3.68  3.82 14.5 10.3  5.16  5     3.68 10.2
9     9 AAM LOW      9.48  6.15  5.96 15.2 12.1  7.52  5.88  3.92  9.75
10    10 AAM INTERM... 10.3  4.75  4.7   14.3 11.1  6.13  4.33  3.11  8.68
# ... with 85 more rows, 51 more variables: CDC20 <dbl>, CDCA3 <dbl>, CDK1 <dbl>,
# CDKN3 <dbl>, CENPF <dbl>, CENPM <dbl>, CEP55 <dbl>, COL1A1 <dbl>,
# DLGAP5 <dbl>, DTL <dbl>, EPPK1 <dbl>, FAM13C <dbl>, FLNC <dbl>,
# FOXM1 <dbl>, GSN <dbl>, GSTM2 <dbl>, IQGAP3 <dbl>, KIAA0101 <dbl>,
# KIF11 <dbl>, KIF20A <dbl>, KLK2 <dbl>, LASP1 <dbl>, MCM10 <dbl>,
# MYBPC1 <dbl>, NFIB <dbl>, NUSAP1 <dbl>, ORC6 <dbl>, PBK <dbl>, PBX1 <dbl>,
# `PCAT-32` <dbl>, `PCAT-80` <dbl>, PCDH7 <dbl>, PLK1 <dbl>, PRC1 <dbl>, ...
# i Use `print(n = ...)` to see more rows, and `colnames()` to see all variable
names
```

Subset Variables (Columns)

Subset Columns



dplyr::select(iris, Sepal.Width, Petal.Length, Species)

Select columns by name or helper function.

Helper functions for select - ?select

select(iris, contains("."))

Select columns whose name contains a character string.

select(iris, ends_with("Length"))

Select columns whose name ends with a character string.

select(iris, everything())

Select every column.

select(iris, matches(".t."))

Select columns whose name matches a regular expression.

select(iris, num_range("x", 1:5))

Select columns named x1, x2, x3, x4, x5.

select(iris, one_of(c("Species", "Genus")))

Select columns whose names are in a group of names.

select(iris, starts_with("Sepal"))

Select columns whose name starts with a character string.

select(iris, Sepal.Length:Petal.Width)

Select all columns between Sepal.Length and Petal.Width (inclusive).

select(iris, -Species)

Select all columns except Species.

Subset rows

Subset Observations (Rows)



dplyr::filter(iris, Sepal.Length > 7)

Extract rows that meet logical criteria.

dplyr::distinct(iris)

Remove duplicate rows.

dplyr::sample_frac(iris, 0.5, replace = TRUE)

Randomly select fraction of rows.

dplyr::sample_n(iris, 10, replace = TRUE)

Randomly select n rows.

dplyr::slice(iris, 10:15)

Select rows by position.

dplyr::top_n(storms, 2, date)

Select and order top n entries (by group if grouped data).

Subset columns and then subset rows

```
> data
# A tibble: 327 × 63
  ID Race CAPRA...1 AN07 ASF1B ASPM AZGP1 BGN BIRC5 BUB1B C6orf10 CAMK2N1
  <dbl> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1     1 AAM INTERM... 10.9 4.11 3.78 13.3 10.6 5.95 4.59 3.78 10.7
2     2 AAM LOW      10.5 6     5.64 13.8 11.5 6.64 5.64 3.41 9.7
3     3 AAM LOW      11.2 4.44 4.63 12.5 10.2 6.63 5.44 3.21 9
4     4 AAM LOW      11.4 4.88 4.19 14.3 10.2 5.54 4.4 3.66 9.65
5     5 AAM LOW      10.5 3.56 4.37 13.0 10.3 4.84 4.56 2.98 9.71
6     6 AAM LOW      10.9 5.98 5.2 14.8 10.9 7.17 5.85 5.06 8.97
7     7 AAM INTERM... 7.54 5.36 4.18 12.4 11.6 5.7 5.03 3.44 10.7
8     8 AAM LOW      11.4 3.68 3.82 14.5 10.3 5.16 5 3.68 10.2
9     9 AAM LOW      9.48 6.15 5.96 15.2 12.1 7.52 5.88 3.92 9.75
10    10 AAM INTERM... 10.3 4.75 4.7 14.3 11.1 6.13 4.33 3.11 8.68
# ... with 317 more rows, 51 more variables: CDC20 <dbl>, CDCA3 <dbl>,
# CDK1 <dbl>, CDKN3 <dbl>, CENPF <dbl>, CENPM <dbl>, CEP55 <dbl>,
# COL1A1 <dbl>, DLGAP5 <dbl>, DTL <dbl>, EPPK1 <dbl>, FAM13C <dbl>,
# FLNC <dbl>, FOXM1 <dbl>, GSN <dbl>, GSTM2 <dbl>, IQGAP3 <dbl>,
# KIAA0101 <dbl>, KIF11 <dbl>, KIF20A <dbl>, KLK2 <dbl>, LASP1 <dbl>,
# MCM10 <dbl>, MYBPC1 <dbl>, NFIB <dbl>, NUSAP1 <dbl>, ORC6 <dbl>, PBK <dbl>,
# PBX1 <dbl>, `PCAT-32` <dbl>, `PCAT-80` <dbl>, PCDH7 <dbl>, PLK1 <dbl>, ...
# i Use `print(n = ...)` to see more rows, and `colnames()` to see all variable
names
> tbl <- select(data, ID, Race, CAPRASgroup)
> filter(tbl, Race == "AAM")
# A tibble: 95 × 3
  ID Race CAPRASgroup
  <dbl> <chr> <chr>
1     1 AAM INTERMEDIATE
2     2 AAM LOW
3     3 AAM LOW
4     4 AAM LOW
5     5 AAM LOW
6     6 AAM LOW
7     7 AAM INTERMEDIATE
8     8 AAM LOW
9     9 AAM LOW
10    10 AAM INTERMEDIATE
# ... with 85 more rows
# i Use `print(n = ...)` to see more rows
```

Can we do it any other way?

- All tidyverse functions take a tibble as their first argument
- All tidyverse functions return a tibble
- You can therefore chain operations together, passing the output of one function as the first input to another

Data → Filter 1 → Filter 2 → Selection

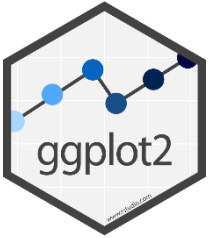
The pipe operator: %>%

- Takes the data on its left and makes it the first argument to a function on its right.

```
[> select(data, ID, Race, CAPRASgroup)
# A tibble: 327 × 3
   ID Race  CAPRASgroup
<dbl> <chr> <chr>
1     1 AAM  INTERMEDIATE
2     2 AAM    LOW
3     3 AAM    LOW
4     4 AAM    LOW
5     5 AAM    LOW
6     6 AAM    LOW
7     7 AAM  INTERMEDIATE
8     8 AAM    LOW
9     9 AAM    LOW
10    10 AAM  INTERMEDIATE
# ... with 317 more rows
# i Use `print(n = ...)`` to see more rows
```

```
[> select(data, ID, Race, CAPRASgroup) %>% filter(Race == "AAM")
# A tibble: 95 × 3
   ID Race  CAPRASgroup
<dbl> <chr> <chr>
1     1 AAM  INTERMEDIATE
2     2 AAM    LOW
3     3 AAM    LOW
4     4 AAM    LOW
5     5 AAM    LOW
6     6 AAM    LOW
7     7 AAM  INTERMEDIATE
8     8 AAM    LOW
9     9 AAM    LOW
10    10 AAM  INTERMEDIATE
# ... with 85 more rows
# i Use `print(n = ...)`` to see more rows
```

Plotting figures and graphs with ggplot



- ggplot is the plotting library for tidyverse
 - Powerful
 - Flexible
- Follows the same conventions as the rest of tidyverse
 - Data stored in tibbles
 - Data is arranged in 'tidy' format
 - Tibble is the first argument to each function

Code structure of a ggplot graph

- Start with a call to `ggplot()`
 - Pass the tibble of data
 - Say which columns you want to use
- Say which graphical representation you want to use
 - Points, lines, barplots etc
- Customise labels, colours annotations etc.

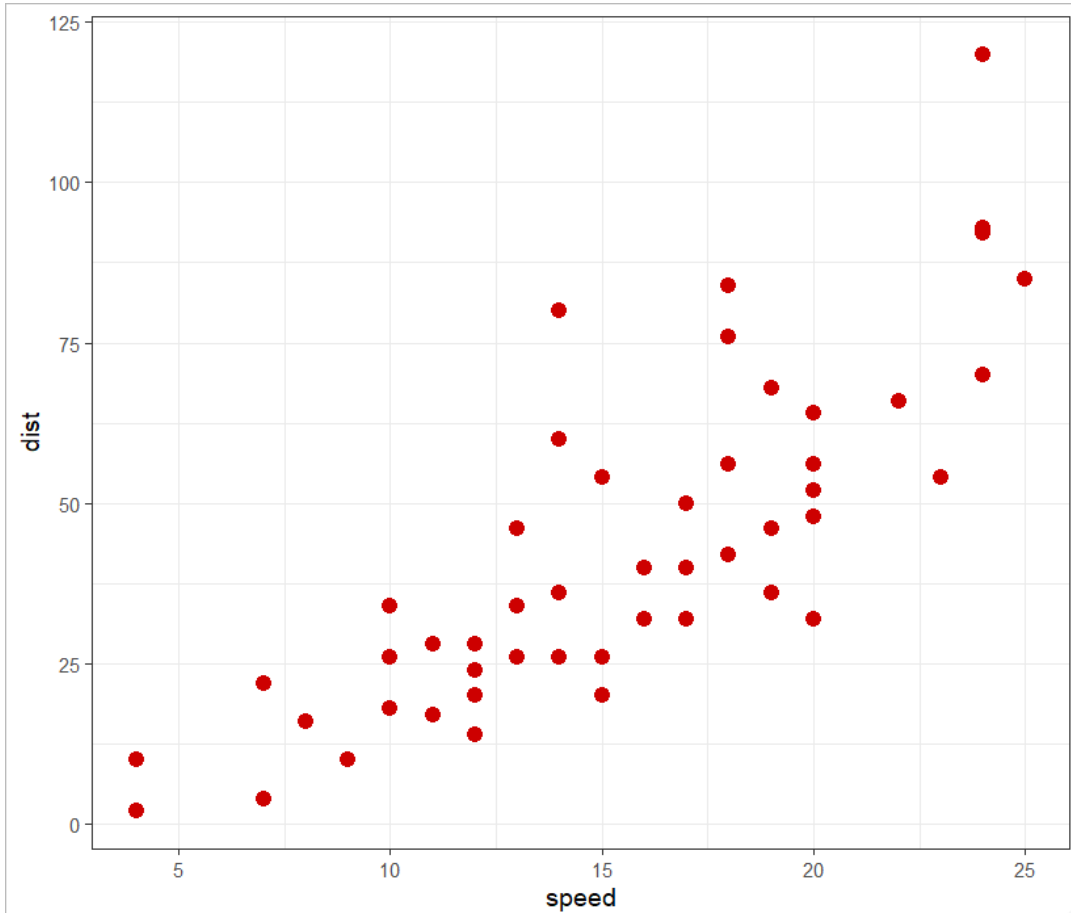
Geometries and Aesthetics

- Geometries are types of plot

<code>geom_point()</code>	Point geometry, (x/y plots, stripcharts etc)
<code>geom_line()</code>	Line graphs
<code>geom_boxplot()</code>	Box plots
<code>geom_bar()</code>	Barplots
<code>geom_histogram()</code>	Histogram plots

- Aesthetics are graphical parameters which can be adjusted in a given geometry

Aesthetics for `geom_point()`



Aesthetics

`geom_point()` understands the following aesthetics (required aesthetics are in bold):

- **x**
- **y**
- alpha
- colour
- **fill**
- group
- shape
- size
- stroke

How do you define aesthetics

- Fixed values
 - Colour all points red
 - Make the points size 4
- Encoded from your data – called an *aesthetic mapping*
 - Colour according to genotype
 - Size based on the number of observations
- Aesthetic mappings are set using the `aes()` function, normally as an argument to the `ggplot` function

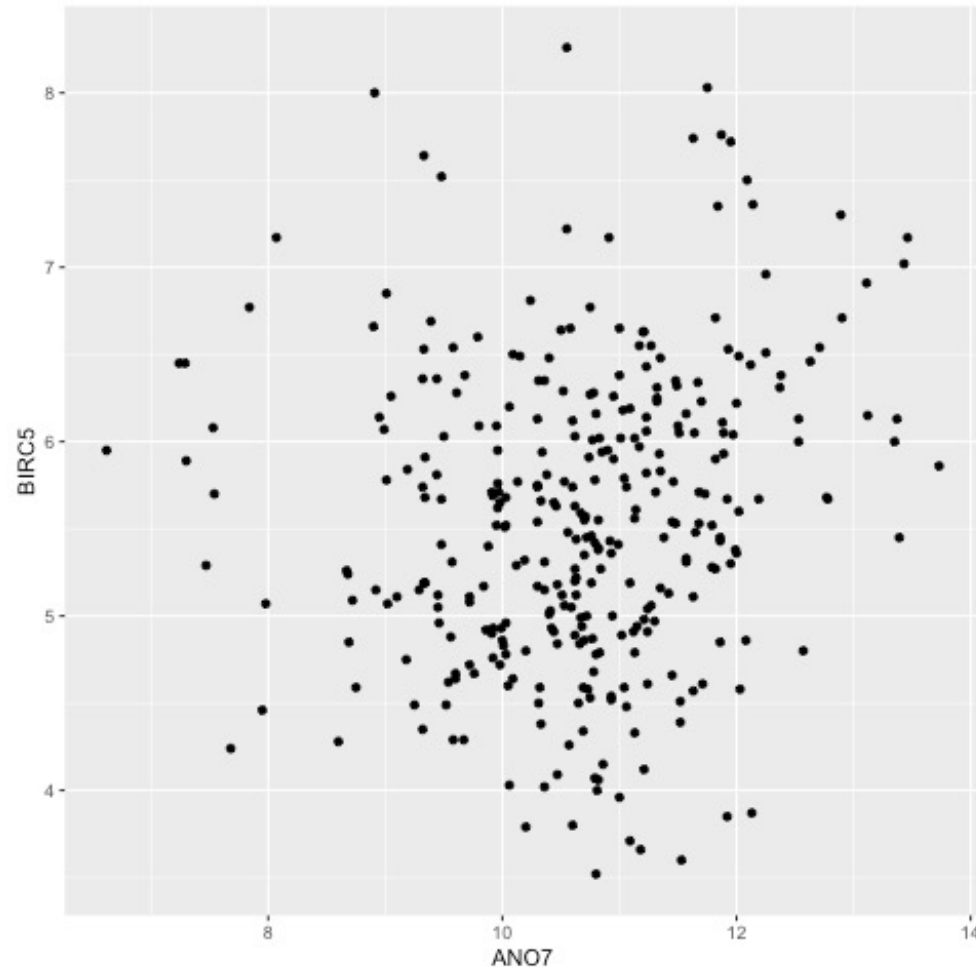
```
data %>% ggplot(aes(x=Race, y=BIRC5, colour=CAPRASgroup))
```

Putting things together

- Identify the tibble with the data you want to plot
 - Decide on the geometry (plot type) you want to use
 - Decide which columns will modify which aesthetic
-
- Call `ggplot(aes(...))`
 - Add a `geom_xxx` function call

Our first plot...

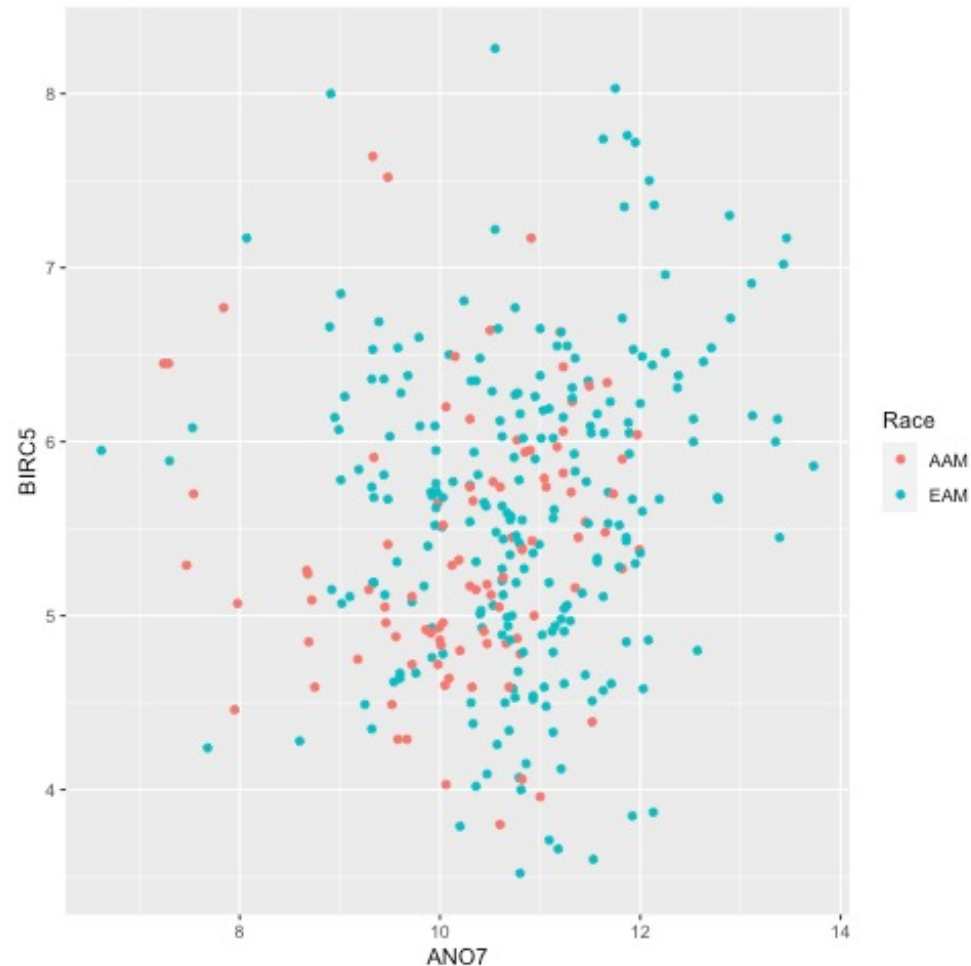
```
ggplot(data , aes (x=ANO7 , y=BIRC5) ) + geom_point ( )
```



- Identify the tibble with the data you want to plot
- Decide on the geometry (plot type) you want to use
- Decide which columns will modify which aesthetic
- Call `ggplot(aes(...))`
- Add a `geom_xxx` function call

Our second plot...

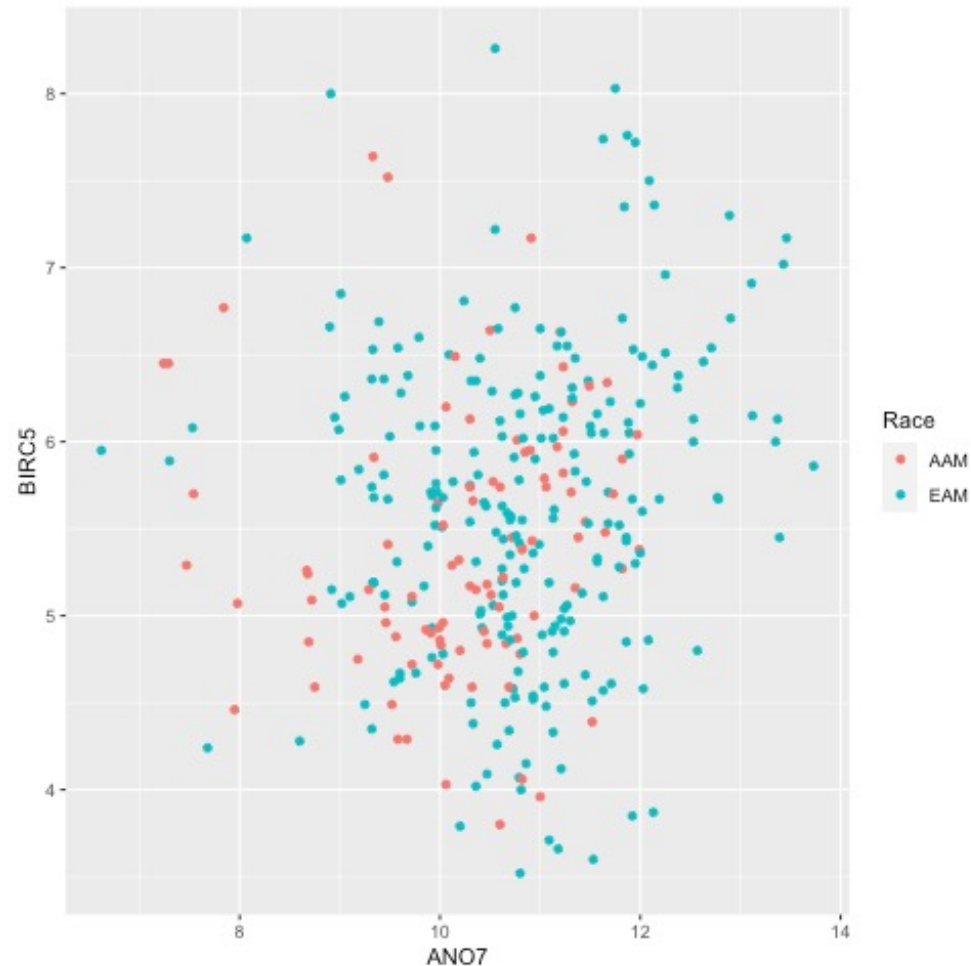
```
ggplot(data, aes(x=ANO7, y=BIRC5, color=Race)) +  
geom_point()
```



- Identify the tibble with the data you want to plot
- Decide on the geometry (plot type) you want to use
- Decide which columns will modify which aesthetic
- Call `ggplot(aes(...))`
- Add a `geom_xxx` function call

Our third plot...

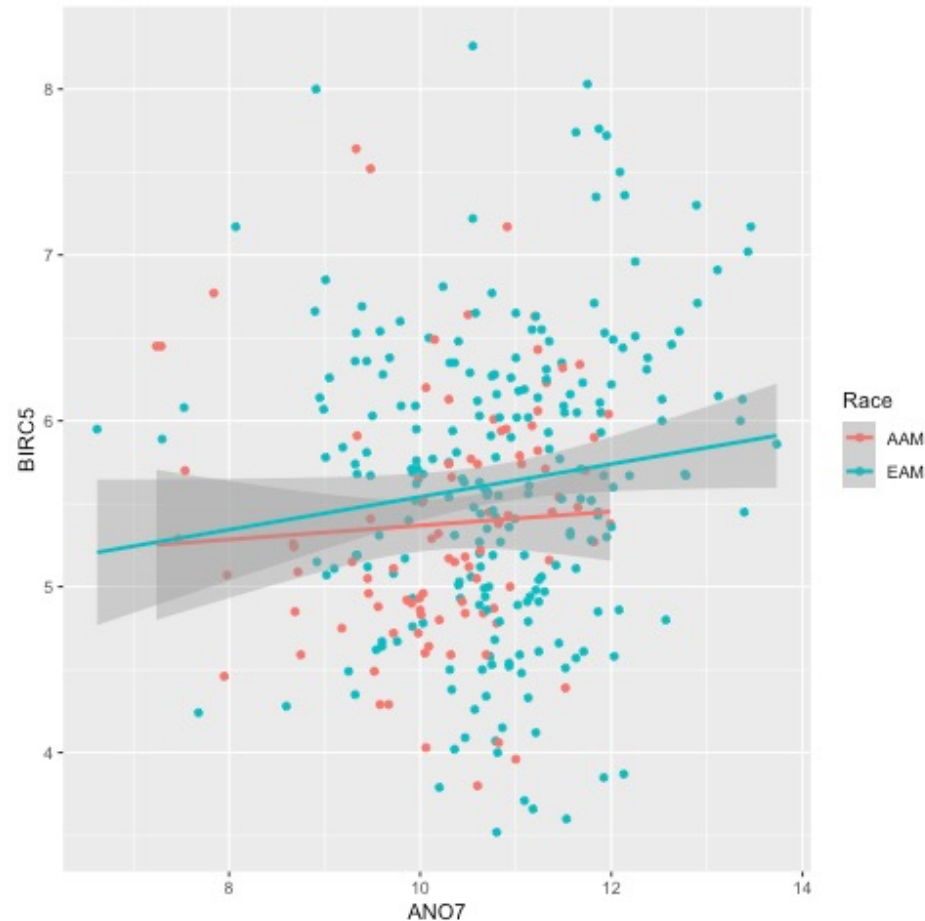
```
data %>% ggplot(aes(x=ANO7, y=BIRC5, color=Race)) +  
geom_point()
```



- Identify the tibble with the data you want to plot
- Decide on the geometry (plot type) you want to use
- Decide which columns will modify which aesthetic
- Call `ggplot(aes(...))`
- Add a `geom_xxx` function call

Our fourth plot...

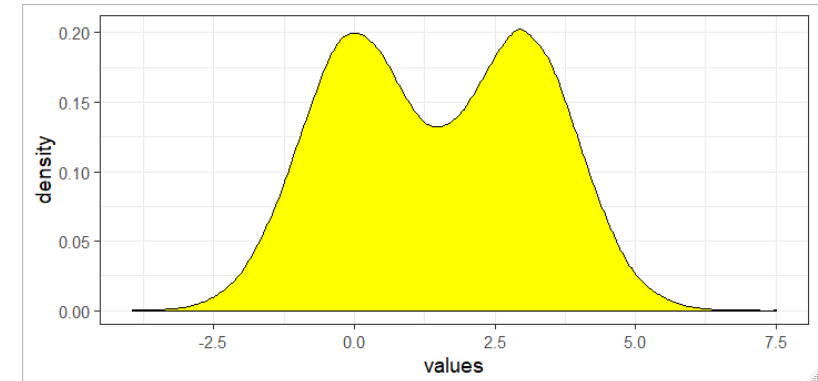
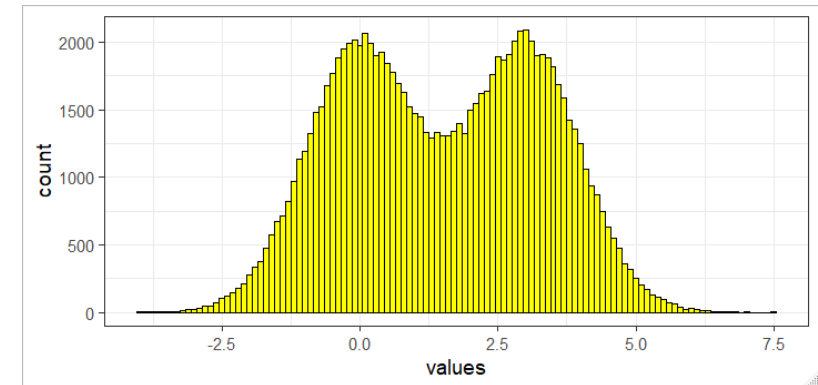
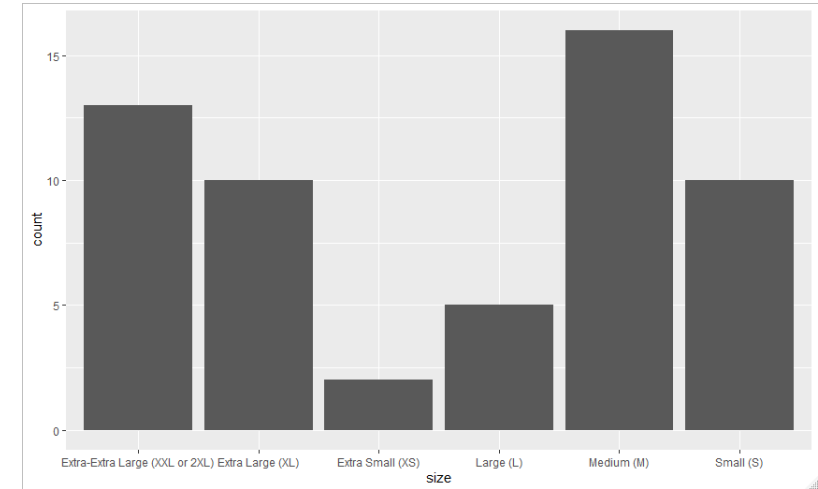
```
data %>% ggplot(aes(x=ANO7, y=BIRC5, color=Race)) +  
  geom_point() + geom_smooth(method="lm")
```



- Identify the tibble with the data you want to plot
- Decide on the geometry (plot type) you want to use
- Decide which columns will modify which aesthetic
- Call `ggplot(aes(...))`
- Add a `geom_xxx` function call

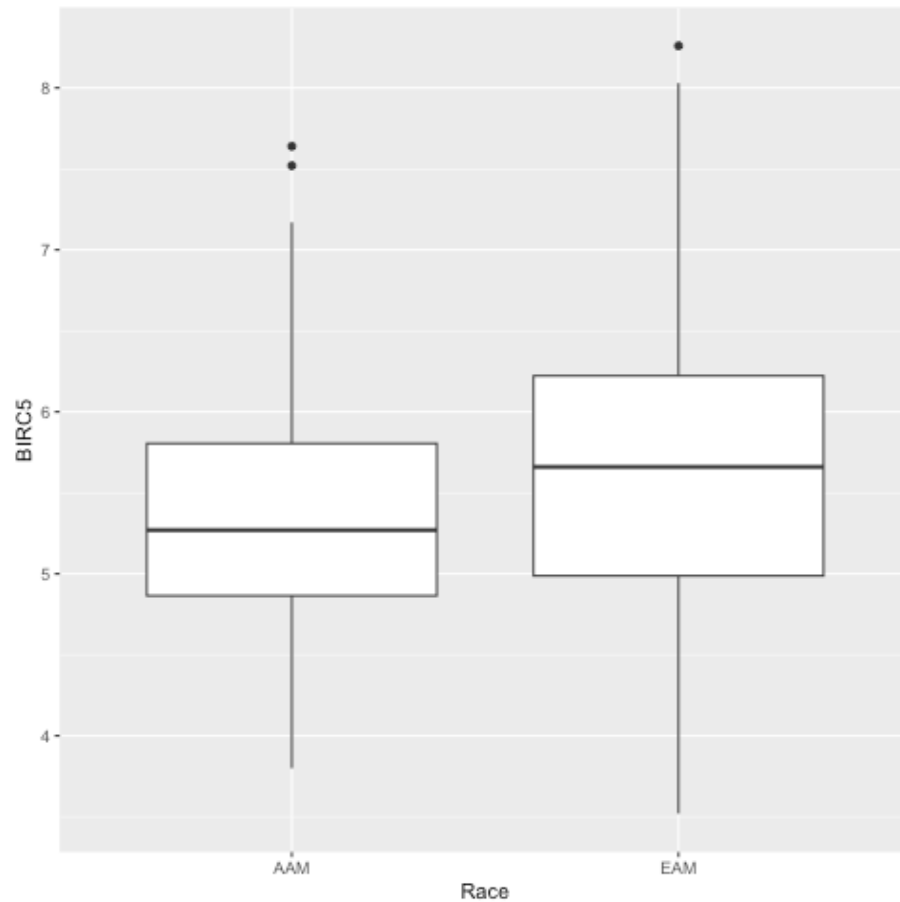
Other plot types

- Barplots
 - `geom_bar`
 - `geom_col`
- Boxplots
 - `geom_boxplot`
- Distribution Plots
 - `geom_histogram`
 - `geom_density`



Boxplot

```
data %>% ggplot(aes(x=Race, y=BIRC5)) +  
  geom_boxplot()
```



- Identify the tibble with the data you want to plot
- Decide on the geometry (plot type) you want to use
- Decide which columns will modify which aesthetic
- Call `ggplot(aes(...))`
- Add a `geom_xxx` function call

And you can keep on improving the plot

```
data %>%  
ggplot(aes(x=Race, y=BIRC5)) +  
  geom_boxplot() +  
  xlab("Race of the patient") +  
  ylab("BIRC5 expression") +  
  ggtitle("BIRC5 expression by race")
```

